FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

1

RANDOM WALK SIMULATION USING AGENTPY

Student Name	Vignesh Lyndon Vinod G. Gopal
Section:	AN43
Professor:	Ms. Crisola Tan
Date:	Oct 14, 2026

I. BACKGROUND INFORMATION

A **random walk** is a mathematical process that describes a path consisting of a sequence of **random steps**. Random walks are used in many fields, such as physics, biology, economics, and computer science, to model systems that evolve randomly over time.

In this laboratory, we simulate multiple agents moving randomly on a grid using **AgentPy**, a Python library for **agent-based modeling**. Each agent represents an entity that can move one step at a time in a random direction. This simulation helps students understand:

- How **agents move and interact** in a grid-based environment.
- The concept of **bounded movement**, ensuring agents stay within the grid limits.
- What changing **parameters** like the number of agents, grid size, or number of steps affects the system.
- How **agent-based models** can be used to simulate real-world phenomena, such as **diffusion, crowd dynamics, or swarm behavior**.

II. TECHNICAL ASSESSMENT

RANDOM WALK SIMULATION USING AGENTPY

Objective

At the end of this laboratory activity, students are expected to:

1. To understand the concept of **random walks** in a grid environment.
2. To implement a **multi-agent simulation** using Python and AgentPy.
3. To visualize the **movement of agents** on a grid over time.

Materials Needed

- Computer with Python 3.x installed
- Libraries: agentpy, matplotlib
- Code editor (e.g., VSCode, PyCharm, or Jupyter Notebook)

General Instructions

1. This **Technical Assessment (TA4)** must be completed individually.
2. Follow the Task below.
3. Include clear comments in your code to describe its logic.
4. Each program should display the correct input, process, and output.
5. Ensure that your script displays clear and readable outputs in the Command Window.
6. Compile all Python codes and corresponding outputs into one PDF file.
7. Save it as **TA1_lastname.pdf**
   (Example: TA1_Patilano.pdf)

8. Submit the **python file** and **PDF** file.

Activity Instructions

Step 1: Run the Code

1. Open the provided Python code.

Random Walk Simulation (Console-Based)

```
import agentpy as ap
import random
import matplotlib.pyplot as plt
import matplotlib.animation as animation

# -------- Get user input from console --------
num_agents = int(input("Enter number of agents: "))
grid_size = int(input("Enter grid size (e.g., 10 for 10x10): "))
num_steps = int(input("Enter number of steps: "))

# -------- Agent Definition -----------
class RandomWalker(ap.Agent):
    def step(self):
        direction = random.choice([(1,0), (-1,0), (0,1), (0,-1)])
        x, y = self.position
        # Keep agents inside grid
        x = max(0, min(self.model.p.grid_size[0]-1, x + direction[0]))
        y = max(0, min(self.model.p.grid_size[1]-1, y + direction[1]))
        self.position = (x, y)

# -------- Model Definition -----------
class RandomWalkModel(ap.Model):
    def setup(self):
        # Create agents
        self.agents = ap.AgentList(self, self.p.agents, RandomWalker)

        # Initialize random positions
        for agent in self.agents:
            agent.position = (random.randint(0, self.p.grid_size[0]-1),
                              random.randint(0, self.p.grid_size[1]-1))

        # Create grid (optional)
        self.grid = ap.Grid(self, self.p.grid_size, torus=False)
        self.grid.add_agents(self.agents)

    def step(self):
        for agent in self.agents:
            agent.step()

# -------- Parameters from user input -----------
parameters = {
    'agents': num_agents,
    'grid_size': (grid_size, grid_size),
    'steps': num_steps
}
```

```
# -------- Run Model -----------
model = RandomWalkModel(parameters)
model.setup()

# -------- Interactive Animation -----------
fig, ax = plt.subplots()
ax.set_xlim(0, model.p.grid_size[0])
ax.set_ylim(0, model.p.grid_size[1])
ax.set_xticks(range(model.p.grid_size[0]+1))
ax.set_yticks(range(model.p.grid_size[1]+1))
ax.grid(True)

scat = ax.scatter([], [], s=200)  # s = size of agents

def update(frame):
    model.step()
    x = [agent.position[0] for agent in model.agents]
    y = [agent.position[1] for agent in model.agents]
    scat.set_offsets(list(zip(x, y)))
    return scat,

ani = animation.FuncAnimation(fig, update, frames=model.p.steps, blit=True, repeat=False)
plt.show()
```

2. Run the program in your terminal.
3. Enter the following values when prompted:
    - **Number of agents** (e.g., 5)
    - **Grid size** (e.g., 10 for a 10x10 grid)
    - **Number of steps** (e.g., 20)

These values control the number of agents, the size of the environment, and the simulation duration.

Step 2: Understand the Agent Behavior

1. Each agent is represented by a dot in the grid.
2. Agents move **randomly** in one of four directions: up, down, left, or right.
3. Agents **cannot leave the grid**; movement is restricted within the grid boundaries.

Step 3: Observe the Simulation

1. After running, a **grid with moving agents** is displayed.
2. Each time step updates the position of all agents.
3. Observe:
    - How agents spread across the grid over time
    - Whether agents cluster in certain areas or move freely

Step 4: Experiment with Parameters

- Repeat the simulation with **different values**:
    - **More agents** (e.g., 10 or 20)
    - **Smaller/larger grid sizes**
    - **Longer simulations** (e.g., 50 steps)

- Observe and record how the **agent movement patterns change**.

Laboratory Tasks

1. **Track agent paths**: Modify the code to **record and display the path of each agent** during the simulation.
2. **Change movement rules**: Make agents **avoid each other** or move in a **preferred direction** and observe the resulting patterns.
3. **Analyze final positions**: Record the **final positions** of all agents and analyze their **distribution**.
4. **Compare scenarios**: Run **two simulations with different parameters** and compare the movement patterns.

Modified (Updated Code - Laboratory Tasks)⇩

```
import agentpy as ap
import random
from collections import Counter
import matplotlib.pyplot as plt
import numpy as np

# -------- Agent Definition -----------
class RandomWalker(ap.Agent):
    def setup(self):
        self.position = (0, 0)
        self.path = [self.position]

    def step(self):
        directions = [(1, 0), (-1, 0), (0, 1), (0, -1)]
        occupied = {other.position for other in self.model.agents
                    if other is not self}

        if self.model.p.preferred_direction:
            preferred = self.model.p.preferred_direction
            directions = [preferred] + [d for d in directions if d != preferred]
        else:
            random.shuffle(directions)

        x, y = self.position
        candidates = []
        for dx, dy in directions:
            candidate = (
                max(0, min(self.model.p.grid_size[0] - 1, x + dx)),
                max(0, min(self.model.p.grid_size[1] - 1, y + dy)),
            )
            if candidate != self.position and (
```

```
                not self.model.p.avoid_agents or candidate not in occupied
            ):
                candidates.append(candidate)

        if candidates:
            self.position = candidates[0]
        self.path.append(self.position)

# -------- Model Definition -----------
class RandomWalkModel(ap.Model):
    def setup(self):
        self.agents = ap.AgentList(self, self.p.agents, RandomWalker)

        positions = set()
        for agent in self.agents:
            position = (random.randrange(self.p.grid_size[0]),
                        random.randrange(self.p.grid_size[1]))
            while self.p.avoid_agents and position in positions:
                position = (random.randrange(self.p.grid_size[0]),
                            random.randrange(self.p.grid_size[1]))
            positions.add(position)
            agent.position = position
            agent.path = [position]

        # Create grid (optional)
        self.grid = ap.Grid(self, self.p.grid_size, torus=False)
        self.grid.add_agents(self.agents)

    def step(self):
        for agent in self.agents:
            agent.step()

def run_scenario(name, parameters):
    model = RandomWalkModel(parameters)
    model.setup()
    for _ in range(parameters["steps"]):
        model.step()
    return name, model

def analyze_final_positions(model):
    positions = [agent.position for agent in model.agents]
    counts = Counter(positions)
    values = np.array(positions)
    print(f"\n{model.p.name} final-position analysis")
```

```
    print("Final positions:", positions)
    print("Unique cells:", len(counts), "of", model.p.grid_size[0] * model.p.grid_size[1])
    print("Most occupied cells:", counts.most_common(5))
    print("Mean position: (%.2f, %.2f)" % tuple(values.mean(axis=0)))
    print("Position standard deviation: (%.2f, %.2f)" % tuple(values.std(axis=0)))
    return counts

def display_comparison(results):
    fig, axes = plt.subplots(2, len(results), figsize=(14, 8), squeeze=False)
    for column, (name, model) in enumerate(results):
        ax_path = axes[0, column]
        ax_final = axes[1, column]
        for index, agent in enumerate(model.agents):
            path = np.array(agent.path)
            ax_path.plot(path[:, 0], path[:, 1], marker=".", linewidth=1,
                         label=f"Agent {index + 1}")
            ax_final.scatter(agent.position[0], agent.position[1], s=80)

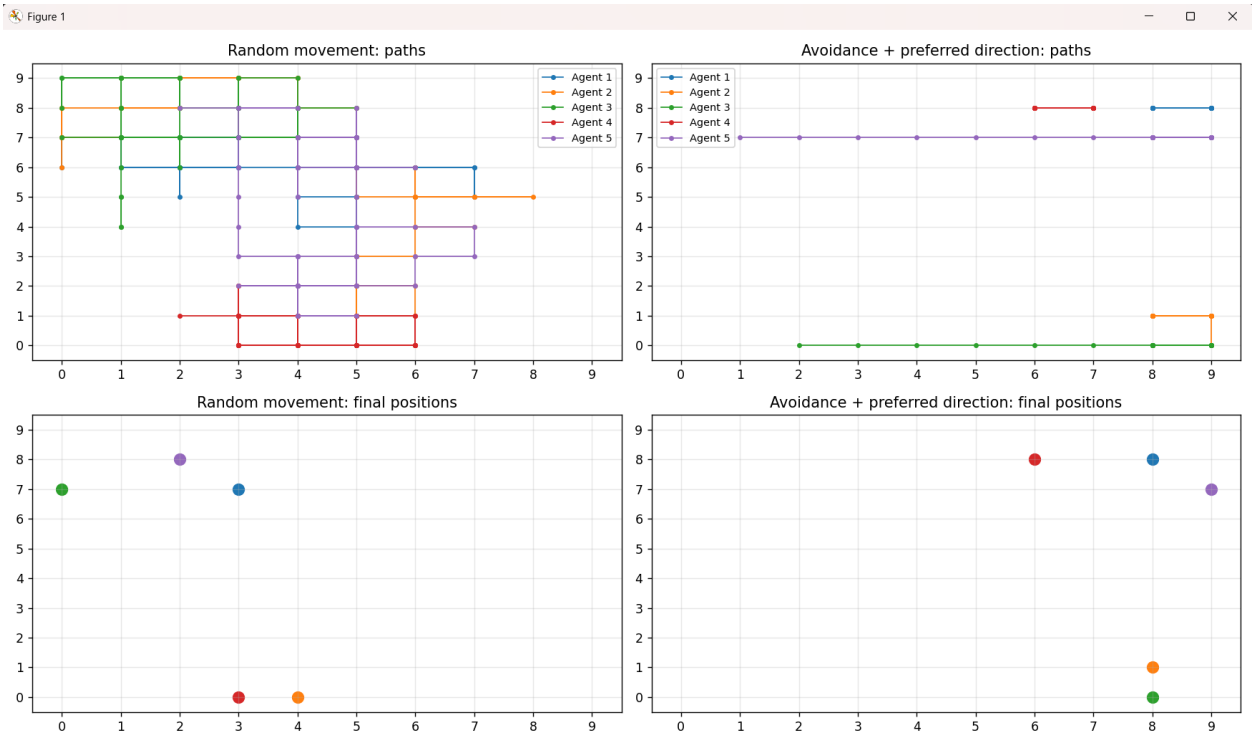
        ax_path.set_title(f"{name}: paths")
        ax_final.set_title(f"{name}: final positions")
        for ax in (ax_path, ax_final):
            ax.set_xlim(-0.5, model.p.grid_size[0] - 0.5)
            ax.set_ylim(-0.5, model.p.grid_size[1] - 0.5)
            ax.set_xticks(range(model.p.grid_size[0]))
            ax.set_yticks(range(model.p.grid_size[1]))
            ax.grid(True, alpha=0.3)
        ax_path.legend(fontsize="small")
    plt.tight_layout()
    plt.show()

num_agents = int(input("Enter number of agents: "))
grid_size = int(input("Enter grid size (e.g., 10 for 10x10): "))
num_steps = int(input("Enter number of steps: "))

common = {
    "agents": num_agents,
    "grid_size": (grid_size, grid_size),
    "steps": num_steps,
}

scenario_random = dict(common, name="Random movement", avoid_agents=False,
                       preferred_direction=None)
scenario_preferred = dict(common, name="Avoidance + preferred direction",
                          avoid_agents=True, preferred_direction=(1, 0))
```

```
results = [run_scenario("Random movement", scenario_random),
           run_scenario("Avoidance + preferred direction", scenario_preferred)]
for _, model in results:
    analyze_final_positions(model)
display_comparison(results)
```



```
PS C:\Programming Projects\4th yr shit> python "C:\Programming Projects\4th yr [redacted]\TA1.py"
Enter number of agents: 5
Enter grid size (e.g., 10 for 10x10): 10
Enter number of steps: 20

Random movement final-position analysis
Final positions: [(2, 8), (0, 3), (5, 8), (2, 4), (4, 4)]
Unique cells: 5 of 100
Most occupied cells: [((2, 8), 1), ((0, 3), 1), ((5, 8), 1), ((2, 4), 1), ((4, 4), 1)]
Mean position: (2.60, 5.40)
Position standard deviation: (1.74, 2.15)

Avoidance + preferred direction final-position analysis
Final positions: [(9, 1), (8, 5), (9, 6), (9, 2), (9, 8)]
Unique cells: 5 of 100
Most occupied cells: [((9, 1), 1), ((8, 5), 1), ((9, 6), 1), ((9, 2), 1), ((9, 8), 1)]
Mean position: (8.80, 4.40)
Position standard deviation: (0.40, 2.58)
PS C:\Programming Projects\4th yr shit> python "C:\Programming Projects\4th yr [redacted]\TA1.py"
Enter number of agents: 5
Enter grid size (e.g., 10 for 10x10): 10
Enter number of steps: 50

Random movement final-position analysis
Final positions: [(3, 7), (4, 0), (0, 7), (3, 0), (2, 8)]
Unique cells: 5 of 100
Most occupied cells: [((3, 7), 1), ((4, 0), 1), ((0, 7), 1), ((3, 0), 1), ((2, 8), 1)]
Mean position: (2.40, 4.40)
Position standard deviation: (1.36, 3.61)

Avoidance + preferred direction final-position analysis
Final positions: [(8, 8), (8, 1), (8, 0), (6, 8), (9, 7)]
Unique cells: 5 of 100
Most occupied cells: [((8, 8), 1), ((8, 1), 1), ((8, 0), 1), ((6, 8), 1), ((9, 7), 1)]
```

Step 5: Answer the Questions

1. What happens if the number of agents is **very high** compared to the grid size?
2. How does increasing the number of steps affect the **spread of agents**?
3. Do the agents ever move outside the grid? Why or why not?
4. How can this simulation be used to **model real-life phenomena** (e.g., diffusion, crowd movement)?
5.

Answer the Questions ⇩

1. The agents still move no problem however, tracking their movement can be hard as it can seem like the agents are overlapping if the number of agents is very high compared to the

grid size. This is reduced in the modified laboratory activity where I added tracking for the agents.

2. Increasing the number of steps usually gives the agents more steps to deviate from their starting point so the result may be more spread out than a low step count run.
3. No the agents do not ever move outside of the grid, this is because of the max() min() variables dictating the maximum grid they are able to travel.
4. This simulation can model real-life phenomena by treating each agent as an individual object that moves according to simple rules.